



INSTITUTO NACIONAL SIMON BOLIVAR

3.3 Retroalimentado con practicas CRUD

NOMBRE : Kevin Alexander Álvarez López

N° LISTA : 1

DOCENTE : Raquel Esperanza Medrano

MODULO 3.3 BASE DE DATOS

Año: 2026

Capturas de PgAdmin

The screenshot shows the PgAdmin interface with a SQL query being executed. The query is as follows:

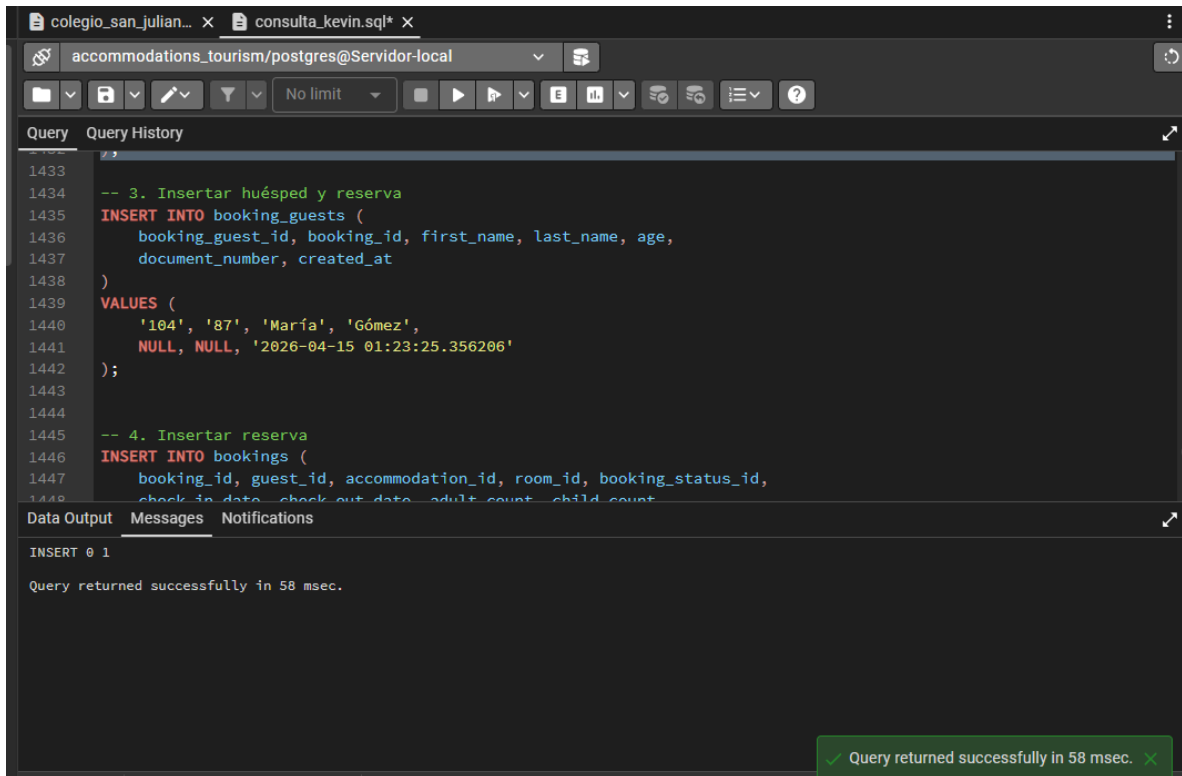
```
1404 phone, lang_id, user_id_email, user_id_email, city,  
1405 state, country, postal_code, created_at, updated_at  
1406 )  
1407 VALUES (  
1408 'Carlos', 'Hernández', 'Servicios Turísticos del Pacífico S.A. de C.V.',  
1409 'carlos.hernandez@gmail.com', '503-7123-4589', 'H-62948153',  
1410 'Calle Los Pinos 24', NULL, 'San Salvador',  
1411 'San Salvador', 'El Salvador', '1101-4582',  
1412 NOW(), NOW()  
1413 );  
1414  
1415 SELECT * FROM owners;  
1416  
1417 -- 2. Insertar alojamiento  
1418 INSERT INTO accommodations (
```

The Data Output tab shows the results of the `SELECT * FROM owners;` query. The output is a table with 7 columns: `owner_id` (PK), `first_name`, `last_name`, `company_name`, `email`, and `phone`. The table contains 5 rows of data.

	owner_id [PK] bigint	first_name character varying (100)	last_name character varying (100)	company_name character varying (150)	email character varying (150)	phone character varying (30)
1	1	Angel	Hierro	Laboratorios Canales-Trejo	fjohnson@example.org	908-386-3794x0265
2	2	Hugo	Bellido	[null]	antoinettevollbrecht@example...	1-648-350-3056x413
3	3	Karolina	Nieto	Graham-Chavez	anunciacioncamino@example.n...	+34982 932 528
4	4	Hermann-Josef	Hartmann	Zimmerman Inc	aparicioalma@example.org	+49(0) 473829973
5	5	Cristina	Leduc	Bahena, de Anda y Ojeda	ybaker@example.com	+34 949872343

Total rows: 22 | Query complete 00:00:00.183 | CRLF | Ln 1415, Col 1

Captura 1 – Inserción de un nuevo propietario (`INSERT INTO owners`) y verificación del registro mediante `SELECT * FROM owners`, donde se listan los 22 propietarios almacenados en la tabla.



The screenshot shows a PostgreSQL query editor with the following SQL code:

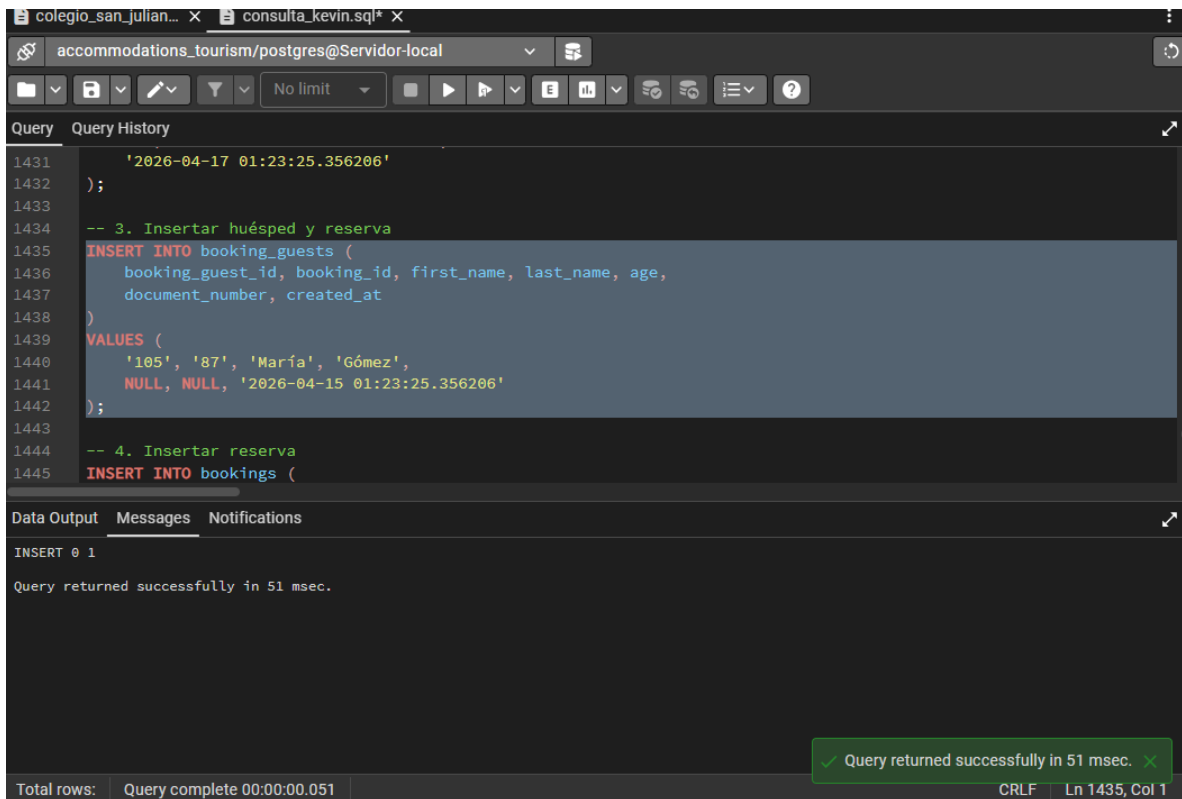
```
-- 3. Insertar huésped y reserva
INSERT INTO booking_guests (
    booking_guest_id, booking_id, first_name, last_name, age,
    document_number, created_at
)
VALUES (
    '104', '87', 'María', 'Gómez',
    NULL, NULL, '2026-04-15 01:23:25.356206'
);
```

The Data Output tab shows the result of the query:

```
INSERT 0 1
Query returned successfully in 58 msec.
```

A green status bar at the bottom right indicates: "Query returned successfully in 58 msec."

Captura 2 – Ejecución de la sentencia `INSERT INTO accommodations` para registrar un nuevo alojamiento; el panel de mensajes confirma “`INSERT 0 1`”, indicando que la inserción se realizó correctamente.



The screenshot shows a PostgreSQL query editor with the following SQL code:

```
'2026-04-17 01:23:25.356206'
);
-- 3. Insertar huésped y reserva
INSERT INTO booking_guests (
    booking_guest_id, booking_id, first_name, last_name, age,
    document_number, created_at
)
VALUES (
    '105', '87', 'María', 'Gómez',
    NULL, NULL, '2026-04-15 01:23:25.356206'
);
-- 4. Insertar reserva
INSERT INTO bookings (
```

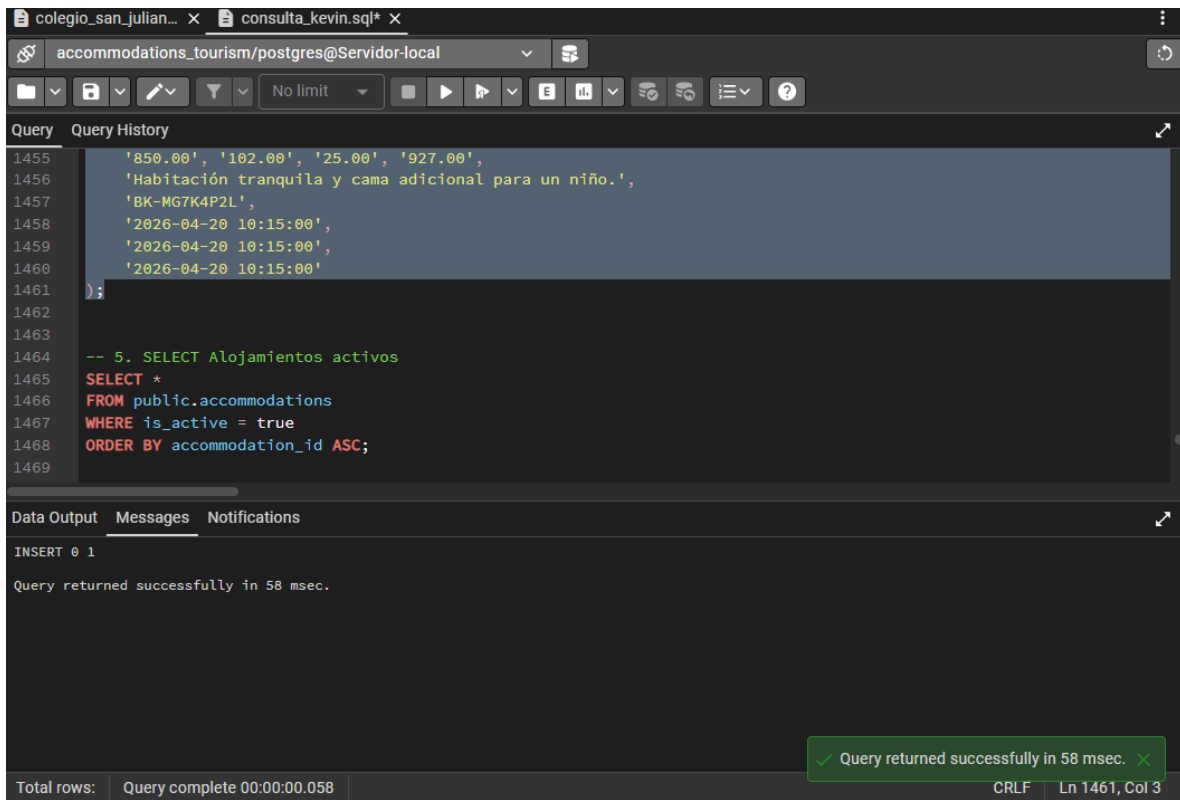
The Data Output tab shows the result of the query:

```
INSERT 0 1
Query returned successfully in 51 msec.
```

A green status bar at the bottom right indicates: "Query returned successfully in 51 msec."

At the bottom of the editor, the status bar shows: "Total rows: Query complete 00:00:00.051 CRLF Ln 1435, Col 1"

Captura 3 – Inserción de un nuevo huésped en la tabla `booking_guests` (`INSERT INTO booking_guests`) asociado a una reserva; el mensaje “INSERT 0 1” confirma que el registro se guardó correctamente.



The screenshot shows a PostgreSQL query editor window with two tabs: `colegio_san_julian...` and `consulta_kevin.sql*`. The active tab is `consulta_kevin.sql*`, showing a query in the `Query` pane. The query is an `INSERT INTO` statement for the `booking_guests` table, followed by a `SELECT` statement to verify the insertion. The `Data Output` pane shows the result of the `INSERT` statement: `INSERT 0 1`, indicating that one row was successfully inserted. A green status bar at the bottom right confirms: `Query returned successfully in 58 msec.`

```
1455 '850.00', '102.00', '25.00', '927.00',
1456 'Habitación tranquila y cama adicional para un niño.',
1457 'BK-MG7K4P2L',
1458 '2026-04-20 10:15:00',
1459 '2026-04-20 10:15:00',
1460 '2026-04-20 10:15:00'
1461 );
1462
1463
1464 -- 5. SELECT Alojamientos activos
1465 SELECT *
1466 FROM public.accommodations
1467 WHERE is_active = true
1468 ORDER BY accommodation_id ASC;
1469
```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 58 msec.

✓ Query returned successfully in 58 msec. ✕

Total rows: Query complete 00:00:00.058 CRLF Ln 1461, Col 3

Captura 4 – Finalización de la sentencia `INSERT INTO bookings` para registrar una nueva reserva, con los datos de precio, descripción, código y fechas; debajo se aprecia el inicio de la consulta 5 (`SELECT` de alojamientos activos).

colegio_san_julian... X consulta_kevin.sql* X

accommodations_tourism/postgres@Servidor-local

Query Query History

```

1480 ORDER BY guest_id ASC;
1481
1482 SELECT *
1483 FROM public.guests
1484 WHERE nationality = 'Japón'
1485 ORDER BY guest_id ASC;
1486
1487
1488 -- 7. SELECT Reservas por rango de fechas
1489 SELECT
1490     booking_id,
1491     check_in_date,
1492     check_out_date
1493 FROM bookings
1494 WHERE check_out_date BETWEEN '2026-04-01'

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

guest_id [PK] bigint	first_name character varying (100)	last_name character varying (100)	email character varying (150)	phone character varying (30)	date_of_birth date	nationality character varying (100)	pas cha
1	20	Corey	Segovia	tilmannkoester@example.c...	+33 4 85 07 14 61	2001-11-08	Japón

Successfully run. Total query runtime: 78 msec. 1 rows affected.

Total rows: 1 Query complete 00:00:00.078 CRLF Ln 1485, Col 23

Captura 5 – Consulta SELECT sobre la tabla guests filtrando por nationality = 'Japón', ordenada por guest_id; el resultado muestra el único huésped japonés registrado.

colegio_san_julian... X consulta_kevin.sql* X

accommodations_tourism/postgres@Servidor-local

Query Query History

```

484 WHERE nationality = 'Japón'
485 ORDER BY guest_id ASC;
486
487
488 -- 7. SELECT Reservas por rango de fechas
489 SELECT
490     booking_id,
491     check_in_date,
492     check_out_date
493 FROM bookings
494 WHERE check_out_date BETWEEN '2026-04-01'
495 AND '2026-07-15';
496
497
498 -- 8. UPDATE Actualizar precio

```

Data Output Messages Notifications

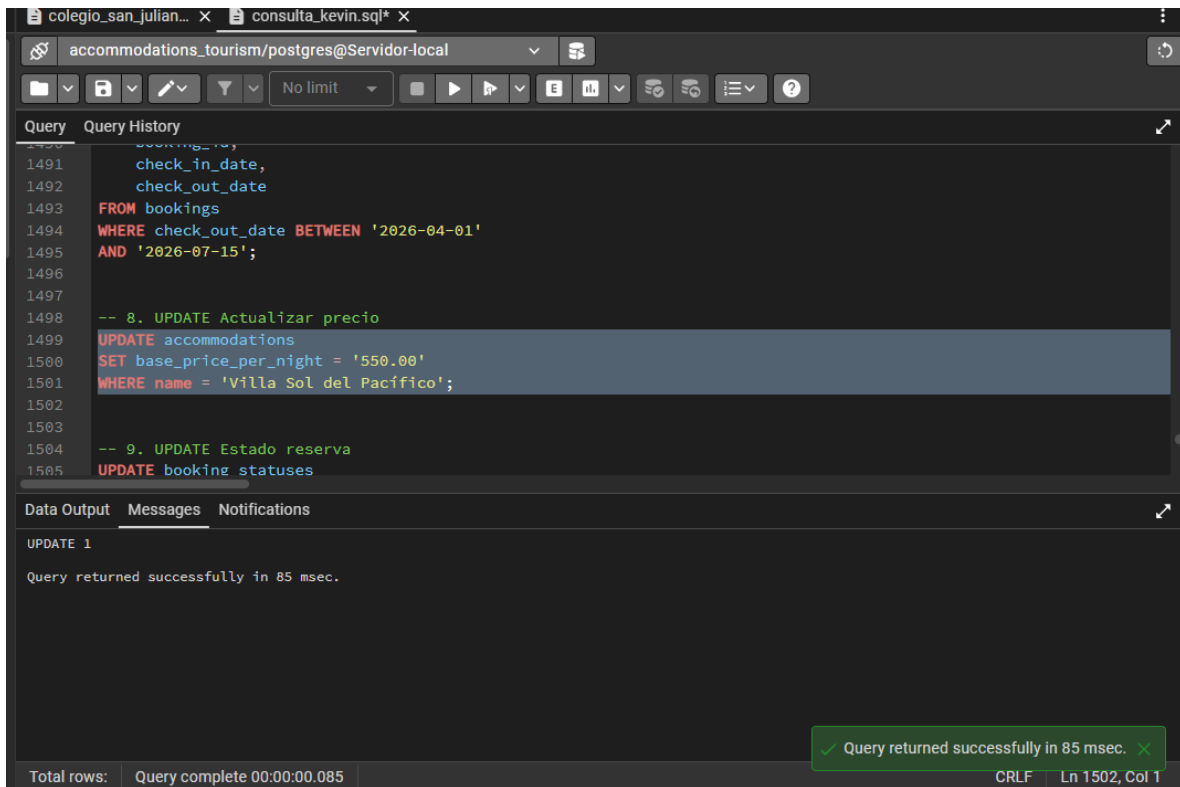
Showing rows: 1 to 15 Page No: 1 of 1

booking_id [PK] bigint	check_in_date date	check_out_date date
6	2026-04-08	2026-04-21
7	2026-05-16	2026-05-26
20	2026-06-17	2026-06-29
23	2026-04-24	2026-05-05
29	2026-04-14	2026-04-22

Successfully run. Total query runtime: 93 msec. 15 rows affected.

Total rows: 15 Query complete 00:00:00.093 CRLF Ln 1496, Col 1

Captura 6 – Consulta *SELECT* sobre la tabla *bookings* que filtra las reservas cuya *check_out_date* está comprendida entre '2026-04-01' y '2026-07-15'; se obtienen 15 filas.



The screenshot shows a PostgreSQL query editor interface. The top toolbar includes icons for file operations, query execution, and settings. The main editor area displays a SQL query with line numbers 1490 to 1505. The query consists of a *SELECT* statement filtering *bookings* by *check_out_date* and two commented-out *UPDATE* statements. The *Data Output* tab shows the message "UPDATE 1" and "Query returned successfully in 85 msec." A green notification bar at the bottom right confirms the successful execution of the query.

```
1490     check_in_date,  
1491     check_out_date  
1492 FROM bookings  
1493 WHERE check_out_date BETWEEN '2026-04-01'  
1494 AND '2026-07-15';  
1495  
1496  
1497  
1498 -- 8. UPDATE Actualizar precio  
1499 UPDATE accommodations  
1500 SET base_price_per_night = '550.00'  
1501 WHERE name = 'Villa Sol del Pacífico';  
1502  
1503  
1504 -- 9. UPDATE Estado reserva  
1505 UPDATE booking statuses
```

UPDATE 1
Query returned successfully in 85 msec.

Query returned successfully in 85 msec. X

Total rows: Query complete 00:00:00.085 CRLF Ln 1502, Col 1

Captura 7 – Ejecución de *UPDATE accommodations* para modificar el *base_price_per_night* del alojamiento 'Villa Sol del Pacífico'; el mensaje "UPDATE 1" confirma que se actualizó un registro.

The screenshot shows a PostgreSQL query editor with the following content:

```
-- 9. UPDATE Estado reserva
UPDATE booking_statuses
SET status_name = 'confirmed'
WHERE booking_status_id = 5;
```

The "Data Output" tab shows the result: "UPDATE 1" and "Query returned successfully in 46 msec." A green status bar at the bottom right confirms: "Query returned successfully in 46 msec."

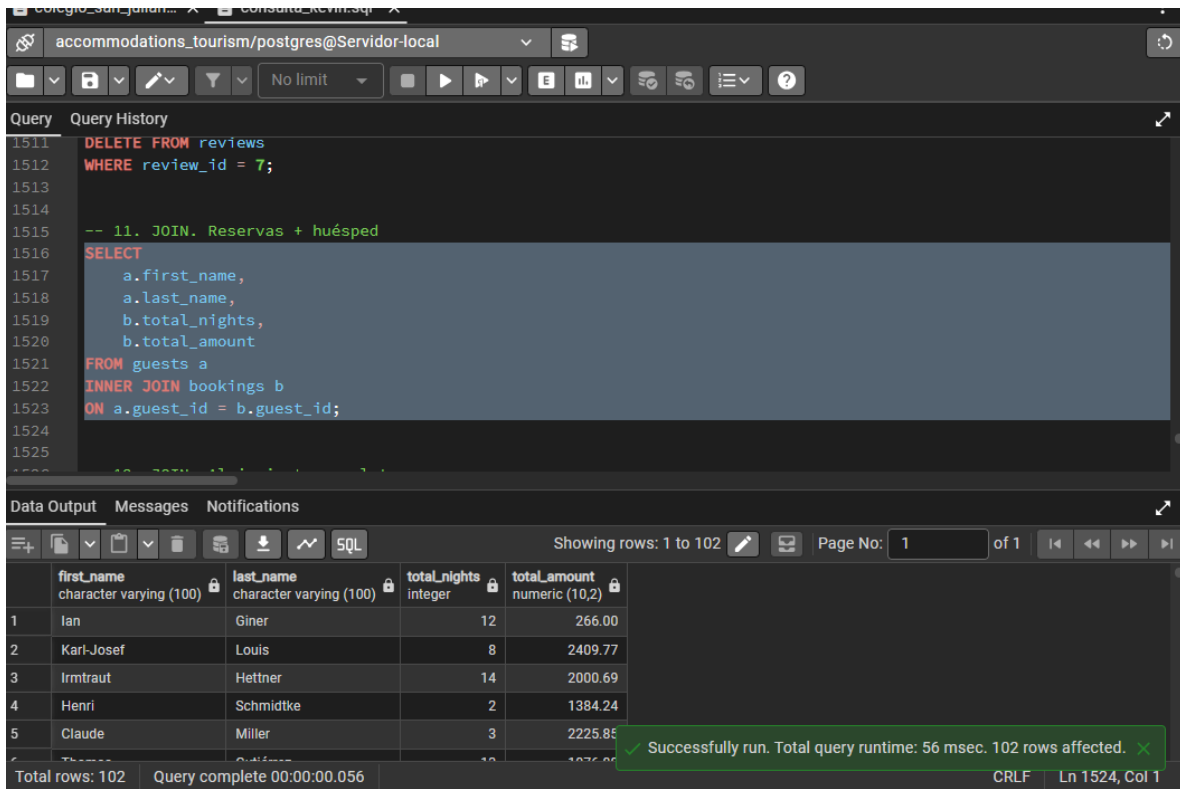
Captura 8 – Ejecución de UPDATE booking_statuses para cambiar el status_name a 'confirmed' en el registro con booking_status_id = 5; el resultado "UPDATE 1" confirma el cambio.

The screenshot shows a PostgreSQL query editor with the following content:

```
-- 10. DELETE. Eliminar reseña
DELETE FROM reviews
WHERE review_id = 7;
```

The "Data Output" tab shows the result: "DELETE 1" and "Query returned successfully in 53 msec."

Captura 9 – Ejecución de `DELETE FROM reviews` para eliminar la reseña con `review_id = 7`; el mensaje “DELETE 1” confirma que el registro fue eliminado.



The screenshot shows a PostgreSQL query editor interface. The top panel displays the query history with the following SQL statements:

```

1511 DELETE FROM reviews
1512 WHERE review_id = 7;
1513
1514
1515 -- 11. JOIN. Reservas + huéspedes
1516 SELECT
1517     a.first_name,
1518     a.last_name,
1519     b.total_nights,
1520     b.total_amount
1521 FROM guests a
1522 INNER JOIN bookings b
1523 ON a.guest_id = b.guest_id;
1524
1525
1526

```

The bottom panel shows the "Data Output" tab with a table of results. The table has 102 rows and 4 columns: `first_name`, `last_name`, `total_nights`, and `total_amount`. The first five rows are visible:

	first_name	last_name	total_nights	total_amount
1	Ian	Giner	12	266.00
2	Karl-Josef	Louis	8	2409.77
3	Irmtraut	Hettner	14	2000.69
4	Henri	Schmidtke	2	1384.24
5	Claude	Miller	3	2225.85

A green status bar at the bottom indicates: "Successfully run. Total query runtime: 56 msec. 102 rows affected." The bottom status bar shows "Total rows: 102" and "Query complete 00:00:00.056".

Captura 10 – Consulta con `INNER JOIN` entre `guests` y `bookings` para mostrar el nombre del huésped junto con el total de noches y el monto total de cada reserva; se obtienen 102 filas.

colegio_san_julian... X consulta_kevin.sql* X

accommodations_tourism/postgres@Servidor-local

Query Query History

```

1533     a.max_guests AS max_huespedes,
1534     a.bedroom_count AS habitaciones,
1535     a.bathroom_count AS sanitarios,
1536     a.check_in_time AS hora_entrada,
1537     a.check_out_time AS hora_salida,
1538     a.is_active AS activo
1539 FROM accommodations AS a
1540 INNER JOIN accommodation_types AS b
1541     ON a.accommodation_type_id = b.accommodation_type_id
1542 WHERE a.is_active = TRUE
1543 ORDER BY a.name ASC;
1544
1545
1546 -- 13. JOIN. Pagos + reserva
1547 SELECT

```

Data Output Messages Notifications

Showing rows: 1 to 19 Page No: 1 of 1

	accommodation_id bigint	alojamiento character varying (150)	tipo character varying (50)	precio_noche numeric (10,2)	moneda character (3)	max_huespedes integer	habitaciones integer	sanitarios integer	horario time
1	9	Boutique Nest boeuf	Guesthouse	445.08	GBP	4	1	1	14
2	3	Boutique Retreat Ville	Resort	230.50	BRL	12	6	3	13
3	11	Boutique Stay Ville	House	142.75	USD	1	6	1	13
4	19	Boutique Suite los bajos	House	378.70	MXN	1	1	4	12
5	18	Charming Stay los bajos	Hotel						

Successfully run. Total query runtime: 76 msec. 19 rows affected.

Total rows: 19 Query complete 00:00:00.076 CRLF Ln 1543, Col 21

Captura 11 – Consulta con INNER JOIN entre accommodations y accommodation_types que lista los alojamientos activos junto con su tipo, capacidad, habitaciones y horarios; se obtienen 19 filas.

colegio_san_julian... X consulta_kevin.sql* X

accommodations_tourism/postgres@Servidor-local

Query Query History

```

1560     INNER JOIN accommodations AS a
1561         ON b.accommodation_id = a.accommodation_id
1562     INNER JOIN payments AS p
1563         ON p.booking_id = b.booking_id
1564     ORDER BY b.check_in_date DESC;
1565
1566
1567 -- 14. LEFT JOIN. Alojamientos sin reseñas
1568 SELECT
1569     a.accommodation_id,
1570     a.name AS alojamiento,
1571     r.review_id,
1572     r.rating,
1573     r.review_title
1574 FROM accommodations AS a

```

Data Output Messages Notifications

Showing rows: 1 to 90 Page No: 1 of 1

	reserva_id bigint	entrada date	salida date	total_reserva numeric (10,2)	estado_reserva integer	alojamiento character varying (150)	pago_id bigint	monto_pagado numeric (10,2)	fecha_pago timestamp without time zone
1	60	2026-07-...	2026-07-...	3016.65	4	Rustic Lodge Ville	52	2411.33	2026-04-15 01:23:25.356206
2	85	2026-07-...	2026-07-...	364.39	4	Modern Escape Ville	75	813.05	2026-04-15 01:23:25.356206
3	50	2026-06-...	2026-07-...	1515.84	6	Luxury Nest Ville	43	1894.02	2026-04-15 01:23:25.356206
4	64	2026-06-...	2026-06-...	107.73	6	Charming Stay los bajos	56	2431.66	2026-04-15 01:23:25.356206
5	20	2026-06-...	2026-06-...	2149.33	6	Panora			

Successfully run. Total query runtime: 89 msec. 90 rows affected.

Total rows: 90 Query complete 00:00:00.089 CRLF Ln 1564, Col 31

Captura 12 – Consulta que combina bookings, accommodations y payments mediante INNER JOIN para mostrar cada reserva con su alojamiento y el pago asociado, ordenada por fecha de entrada; se obtienen 90 filas.

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'accommodations_tourism/postgres@Servidor-local'. The query editor contains the following SQL code:

```

1570     a.name AS alojamiento,
1571     r.review_id,
1572     r.rating,
1573     r.review_title
1574 FROM accommodations AS a
1575 LEFT JOIN reviews AS r
1576     ON a.accommodation_id = r.accommodation_id
1577 WHERE r.review_id IS NULL
1578 ORDER BY a.name ASC;
1579
1580
1581 -- 15. LEFT JOIN. Alojamientos sin reservas
1582 SELECT
1583     a.accommodation_id,
1584     a.name AS alojamiento,

```

Below the query editor, the 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with the following columns: accommodation_id, alojamiento, review_id, rating, and review_title. The table shows 2 rows of data.

	accommodation_id bigint	alojamiento character varying (150)	review_id bigint	rating integer	review_title character varying (150)
1	21	Panoramic Getaway dan	[null]	[null]	[null]
2	22	Villa Sol del Pacifico	[null]	[null]	[null]

At the bottom of the interface, a green status bar indicates: 'Successfully run. Total query runtime: 70 msec. 2 rows affected.' The bottom status bar shows 'Total rows: 2', 'Query complete 00:00:00.070', and 'CRLF Ln 1579, Col 1'.

Captura 13 – Consulta con LEFT JOIN entre accommodations y reviews filtrando WHERE review_id IS NULL, para identificar los alojamientos que aún no tienen reseñas; el resultado muestra 2 alojamientos.

colegio_san_julian... X consulta_kevin.sql* X

accommodations_tourism/postgres@Servidor-local

Query Query History

```
1584     a.name AS alojamiento,
1585     a.base_price_per_night AS precio_noche,
1586     b.booking_id
1587 FROM accommodations AS a
1588 LEFT JOIN bookings AS b
1589     ON a.accommodation_id = b.accommodation_id
1590 WHERE b.booking_id IS NULL
1591 ORDER BY a.name ASC;
1592
1593
1594 -- 16. AGG. Total ingresos
1595 SELECT
1596     SUM(total_amount) AS total_reservaciones
1597 FROM bookings;
1598
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	accommodation_id bigint	alojamiento character varying (150)	precio_noche numeric (10,2)	booking_id bigint
1	21	Panoramic Getaway dan	205.35	[null]
2	22	Villa Sol del Pacifico	550.00	[null]

Successfully run. Total query runtime: 66 msec. 2 rows affected.

Total rows: 2 Query complete 00:00:00.066 CRLF Ln 1591, Col 21

Captura 14 – Consulta con LEFT JOIN entre accommodations y bookings filtrando WHERE booking_id IS NULL, para encontrar los alojamientos sin reservas; se obtienen 2 registros.

The screenshot shows a PostgreSQL query editor interface. The top toolbar includes icons for file operations, query execution, and settings. The main editor area contains the following SQL code:

```
-- 16. AGG. Total ingresos
SELECT
  SUM(total_amount) AS total_reservaciones
FROM bookings;
```

Below the query editor, the 'Data Output' tab is active, displaying a single row of results:

	total_reservaciones numeric
1	176175.42

A green status bar at the bottom indicates: 'Successfully run. Total query runtime: 78 msec. 1 rows affected.'

Captura 15 – Consulta de agregación `SUM(total_amount)` sobre la tabla `bookings` para calcular el total de ingresos por reservaciones; el resultado obtenido es 176175.42.

colegio_san_julian... x consulta_kevin.sql* x

accommodations_tourism/postgres@Servidor-local

Query Query History

```

1592
1593
1594 -- 16. AGG. Total ingresos
1595 SELECT
1596     SUM(total_amount) AS total_reservaciones
1597 FROM bookings;
1598
1599
1600 -- 17. AGG. Promedio rating
1601 SELECT
1602     ROUND(AVG(rating), 2) AS promedio_rating
1603 FROM reviews;
1604
1605
1606 -- 18. AGG. Top alojamientos

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	promedio_rating
	numeric
1	3.14

Total rows: 1 Query complete 00:00:00.063 CRLF Ln 1601, Col 1

Captura 16 – Consulta de agregación $\text{ROUND}(\text{AVG}(\text{rating}), 2)$ sobre la tabla reviews para obtener el promedio de calificaciones; el resultado obtenido es 3.14.

colegio_san_julian... x consulta_kevin.sql* x

accommodations_tourism/postgres@Servidor-local

Query Query History

```

1608     a.accommodation_id,
1609     a.name AS alojamiento,
1610     COUNT(b.booking_id) AS total_reservas
1611 FROM accommodations AS a
1612 INNER JOIN bookings AS b
1613     ON a.accommodation_id = b.accommodation_id
1614 GROUP BY a.accommodation_id, a.name
1615 ORDER BY total_reservas DESC
1616 LIMIT 5;
1617
1618
1619 -- 19. HAVING. Más de 3 reservas
1620 SELECT
1621     a.accommodation_id,
1622     a.name AS alojamiento,

```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	accommodation_id [PK] bigint	alojamiento character varying (150)	totalReservas bigint
1	18	Charming Stay los bajos	9
2	5	Panoramic Stay Ville	8
3	8	Cozy Lodge Ville	7
4	11	Boutique Stay Ville	6
5	15	Luxury Nest Ville	6

✓ Successfully run. Total query runtime: 78 msec. 5 rows affected. ✗

Total rows: 5 Query complete 00:00:00.078 CRLF Ln 1618, Col 1

Captura 17 – Consulta con COUNT, INNER JOIN, GROUP BY, ORDER BY y LIMIT 5 para mostrar el top 5 de alojamientos con más reservas; “Charming Stay los bajos” encabeza la lista con 9 reservas.

The screenshot shows a PostgreSQL IDE window with the following SQL query:

```

1619 -- 19. HAVING. Mas de 3 reservas
1620 SELECT
1621     a.accommodation_id,
1622     a.name AS alojamiento,
1623     COUNT(b.booking_id) AS total_reservas
1624 FROM accommodations AS a
1625 INNER JOIN bookings AS b
1626     ON a.accommodation_id = b.accommodation_id
1627 GROUP BY a.accommodation_id, a.name
1628 HAVING COUNT(b.booking_id) > 3
1629 ORDER BY total_reservas DESC;
1630
1631
1632 -- 20. Subconsulta. Alojamiento más caro
1633 SELECT
1634     accommodation_id

```

The results pane shows the following data:

accommodation_id [PK] bigint	alojamiento character varying (150)	total_reservas bigint
18	Charming Stay los bajos	9
5	Panoramic Stay Ville	8
8	Cozy Lodge Ville	7
15	Luxury Nest Ville	6
11	Boutique Stay Ville	6

A status message at the bottom indicates: "Successfully run. Total query runtime: 163 msec. 19 rows affected."

Captura 18 – Consulta con GROUP BY y HAVING COUNT(b.booking_id) > 3 para filtrar únicamente los alojamientos con más de tres reservas, ordenados de mayor a menor; se obtienen 19 resultados.

The screenshot shows a PostgreSQL query editor with the following SQL code:

```

GROUP BY a.accommodation_id, a.name
HAVING COUNT(b.booking_id) > 3
ORDER BY total_reservas DESC;

-- 20. Subconsulta. Alojamiento más caro
SELECT
    accommodation_id,
    name AS alojamiento,
    base_price_per_night AS precio_noche
FROM accommodations
WHERE base_price_per_night = (
    SELECT MAX(base_price_per_night)
    FROM accommodations
);

```

The Data Output section shows the following result:

	accommodation_id [PK] bigint	alojamiento character varying (150)	precio_noche numeric (10,2)
1	13	Panoramic Suite Ville	597.44

A status message at the bottom indicates: "Successfully run. Total query runtime: 76 msec. 1 rows affected."

Captura 19 – Consulta con subconsulta (*SELECT MAX(base_price_per_night)*) para encontrar el alojamiento más caro; el resultado indica que “Panoramic Suite Ville” tiene el precio más alto, 597.44.

16. AGG — Total ingresos: SUM

Comando: SUM()

Sintaxis:

SELECT SUM(nombre_columna) AS total

FROM nombre_tabla;

¿Para qué sirve?

SUM() es una función de agregación de SQL que sirve para **sumar los valores de una columna numérica** y obtener un total. Se utiliza para calcular cantidades acumuladas, como ingresos, ventas, pagos o costos. Por ejemplo, en una tabla de reservas se puede utilizar SUM(total_amount) para conocer cuánto dinero se ha generado en total. También puede combinarse con GROUP BY para obtener la suma separada por cada alojamiento.

17. AGG — Promedio rating: AVG

Comando: AVG()

Sintaxis:

```
SELECT AVG(nombre_columna) AS promedio  
  
FROM nombre_tabla;
```

¿Para qué sirve?

AVG() es una función de agregación que sirve para **calcular el promedio o media aritmética** de los valores de una columna. Se utiliza principalmente con datos numéricos, como calificaciones, precios, puntuaciones o cantidades. En una base de datos de alojamientos puede utilizarse AVG(rating) para conocer la calificación promedio que han recibido los alojamientos. PostgreSQL calcula el promedio considerando los valores que no sean NULL.

18. AGG — Top alojamientos: COUNT + LIMIT

Comandos: COUNT(), GROUP BY, ORDER BY y LIMIT

Sintaxis:

```
SELECT columna, COUNT(*) AS cantidad  
  
FROM nombre_tabla  
  
GROUP BY columna  
  
ORDER BY cantidad DESC  
  
LIMIT cantidad;
```

¿Para qué sirve?

COUNT() sirve para **contar registros**, GROUP BY permite agruparlos según una columna, ORDER BY permite ordenar los resultados y LIMIT limita la cantidad de resultados que se muestran. Al combinar estos comandos podemos obtener un **Top de alojamientos según la cantidad de reservas**. Por ejemplo, se pueden contar las reservas de cada alojamiento, ordenarlas de mayor a menor y mostrar solamente los primeros cinco. PostgreSQL define COUNT() como una función de agregación y LIMIT como una forma de limitar las filas devueltas por una consulta.

19. HAVING — Más de 3 reservas

Comandos: GROUP BY + HAVING

Sintaxis:

```
SELECT columna, COUNT(*) AS cantidad
```

```
FROM nombre_tabla
```

```
GROUP BY columna
```

```
HAVING COUNT(*) > 3;
```

¿Para qué sirve?

GROUP BY sirve para **agrupar registros que tienen el mismo valor**, mientras que HAVING sirve para **filtrar los grupos obtenidos después de realizar la agrupación**. En este caso, HAVING COUNT(*) > 3 permite mostrar únicamente los alojamientos que tengan más de tres reservas. La diferencia principal con WHERE es que WHERE filtra registros individuales antes de agrupar, mientras que HAVING filtra los grupos después de utilizar GROUP BY.

20. Subconsulta — Alojamiento más caro

Comando: Subconsulta (Subquery) + MAX()

Sintaxis:

```
SELECT *
```

```
FROM nombre_tabla
```

```
WHERE columna = (
```

```
    SELECT MAX(columna)
```

```
    FROM nombre_tabla
```

```
);
```

¿Para qué sirve?

Una **subconsulta** es una consulta SQL que se encuentra dentro de otra consulta y permite utilizar el resultado de la consulta interna en la consulta principal. En este caso, MAX() obtiene primero el valor máximo de una columna, por ejemplo, el precio más alto de los alojamientos. Después, la consulta principal utiliza ese resultado para encontrar el

alojamiento que tiene ese precio. Las subconsultas son útiles cuando necesitamos realizar una consulta previa para obtener un dato que será utilizado como condición en otra consulta. PostgreSQL muestra este tipo de solución precisamente para obtener el registro asociado a un valor máximo.

Ejemplo:

```
SELECT *  
  
FROM accommodations  
  
WHERE price = (  
    SELECT MAX(price)  
    FROM accommodations  
);
```

Aquí la consulta interna:

```
SELECT MAX(price)  
  
FROM accommodations;
```

obtiene primero **el precio más alto**, y la consulta externa busca el alojamiento cuyo precio sea igual a ese resultado.

